
ORACLE
Academy

Database Programming with PL/SQL

15-3

Using Conditional Compilation

ORACLE
Academy

Objectives

- This lesson covers the following objectives:
  - Describe the benefits of conditional compilation
  - Create and conditionally compile a PL/SQL program containing selection, inquiry, and error directives
  - Create and conditionally compile a PL/SQL program which calls the DBMS_DB_VERSION server-supplied package

Purpose

- Imagine you are creating a presentation as part of a school project
- You create it on your home PC using Microsoft Office 2018, which has some nice new features such as a holographic graph display
- When it's finished, you will present your work to your class at school, but until the day of the presentation you won't know if the classroom PC will have Office 2018 or an older version such as Office 2010, which can't display holographic graphs

Purpose

- You want to include holographic graphs in your presentation, but you don't want it to fail while you are presenting to your class
- Conditional compilation can prevent an embarrassing moment

What is Conditional Compilation?

- Wouldn't it be great if you could somehow create your presentation so that if you show it using Office 2018, the holographic graphs are displayed, but if you show it using Office 2010, the standard graphs are displayed instead?
- That way, it won't fail regardless of the Office version you use, and you automatically get the benefit of the new features if they're available
- You can do exactly this in PL/SQL by using Conditional Compilation

What is Conditional Compilation?

- Conditional Compilation allows you to include some source code in your PL/SQL program that may be compiled or may be ignored (like a comment is ignored), depending on:
  - the values of an initialization parameter
  - the version of the Oracle software you are using
  - the value of a global package constant
  - any other condition that you choose to set
- You control conditional compilation by including directives in your source code. Directives are keywords that start with a single or double dollar sign ($ or $$)

This means you can include new features in your application when it runs in the latest database environment and automatically disable the new features when running the application in an older database environment. You also can activate debugging or tracing statements to run in the development environment and automatically ignore them when running the application at a production site.

Conditional Compilation and Microsoft Office

- You can't really use PL/SQL with Microsoft Office, so this is not a real example, but let's pretend:

```
CREATE OR REPLACE PROCEDURE lets_pretend IS
BEGIN
  ...
  $IF MS_OFFICE_VERSION >= '2018' $THEN
   include_holographics;
  $ELSE
   include_std_graphic;
  $END
  ...
END lets_pretend;
```

- $IF, $THEN, $ELSE and $END are selection directives

Conditional Compilation and Microsoft Office

- Conditional compilation uses selection directives, inquiry directives, and error directives to specify source text for compilation
- The selection directive evaluates static expressions to determine which text should be included in the compilation
- The procedure example called LETS_PRETEND uses a selection directive
- The selection directive is of the form:

```
$IF boolean_static_expression $THEN text
  [ $ELSIF boolean_static_expression $THEN text ]
  [ $ELSE text ]
$END
```

- $IF, $THEN, $ELSE and $END are selection directives

The error directive $ERROR raises a user-defined error and is of the form:

$ERROR varchar2_static_expression $END

The inquiry directive is used to check the compilation environment. An inquiry directive can be predefined or be user-defined.

The inquiry directive is of the form:

inquiry_directive ::= $$id

Conditional Compilation and Oracle Versions

- You can't test which Office version you're using, but you can test which Oracle version you're using
- This is a "real" subprogram:

```
CREATE OR REPLACE FUNCTION lets_be_real
  RETURN NUMBER
  $IF DBMS_DB_VERSION.VERSION >= 11 $THEN
   DETERMINISTIC
  $END
IS BEGIN
  RETURN 17; -- real source code here !
END lets_be_real;

BEGIN
  DBMS_OUTPUT.PUT_LINE('Function returned ' || lets_be_real);
END lets_be_real;
```

ORACLE Academy

PLSQL 15-3
Using Conditional Compilation

Generally, conditional compilation is helpful when you are trying to write a portable program, but the way you do something is different depending on what compiler, operating system, or computer you're using. You could use conditional compilation if you want to compile different versions of your program with different features present in the different versions.

Although the example is a "real" function and you can run this code, it is a very simple function. It receives no parameters and always returns the value 17 as a NUMBER datatype.

Conditional Compilation and Oracle Versions

- Deterministic functions are new in Oracle at Version 11
- This code includes the word DETERMINISTIC if we compile the function on Version 11 or later, and is ignored if we compile on Version 10 or earlier

```
CREATE OR REPLACE FUNCTION lets_be_real
  RETURN NUMBER
  $IF DBMS_DB_VERSION.VERSION >= 11 $THEN
   DETERMINISTIC
  $END
IS BEGIN
  RETURN 17; -- real source code here !
END lets_be_real;
```

ORACLE Academy

To learn more about the DETERMINISTIC option, you can explore what it does and when to use it at http://docs.oracle.com.

Basically, the DETERMINISTIC option speeds up the execution of any code calling this function and passing it the same parameter values as were passed to it in a previous call to this function.

What is in the Data Dictionary Now?

- After compiling the function on the previous slide, what is stored in the Data Dictionary?
- USER_SOURCE contains your complete source code, including the compiler directives:

```
SELECT text FROM USER_SOURCE
  WHERE name =
   UPPER('lets_be_real')
  ORDER BY line;
```

TEXT
FUNCTION lets_be_real
RETURN NUMBER
$IF DBMS_DB_VERSION.VERSION>=11$THEN
DETERMINISTIC
$SEND
IS BEGIN
RETURN 17; -- real source code here!
END lets_be_real;

Seeing Which Code has been Compiled

- If you want to see which code has actually been included in your compiled program, you use the DBMS_PREPROCESSOR Oracle-supplied package:

```
BEGIN
  DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE
  ('FUNCTION','<YOUR_USERNAME>','lets_be_real');
END;
```

- This function was compiled using Oracle Version 11:

```
FUNCTION lets_be_real
  RETURN NUMBER

       DETERMINISTIC

IS BEGIN
  RETURN 17; -- real source code here !
END lets_be_real;
```

Seeing Which Code has been Compiled

- The PRINT_POST_PROCESSED_SOURCE procedure returns the source code text of a stored PL/SQL unit that resulted from the most recent successful compilation of its actual source code
- The basic parameters forPRINT_POST_PROCESSED_SOURCE are:
  - object_type - Must be PACKAGE, PACKAGE BODY, PROCEDURE, FUNCTION, or TRIGGER
  - schema_name - Enter your user name
  - object_name - Enter the name of the object

The DBMS_PREPROCESSOR package has other capabilities you can explore at http://docs.oracle.com.

Using Selection Directives

- There are five selection directives: $IF, $THEN, $ELSE, $ELSIF and $END (not $ENDIF)
- Their logic is the same as IF, THEN, ELSE and so on, but they control which code is included at compile time, not what happens at execution time

```
...
$IF condition $THEN statement(s);
$ELSE  statement(s);
$ELSIF statement(s);
$END ...
```

- Notice that $END does not end with a semicolon(;) unlike END;

Using Selection Directives Example

- You have created a bodiless package that declares a number of global constants:

```
CREATE OR REPLACE PACKAGE tax_code_pack IS
  new_tax_code CONSTANT BOOLEAN := TRUE;
     -- but could be FALSE
  ...
END tax_code_pack;
```

- Now, you want to create a subprogram that declares an explicit cursor whose WHERE clause will depend on the value of the Boolean package constant

Using Selection Directives Example

- Now let's look at the contents of the Data Dictionary
- Remember, the package set NEW_TAX_CODE to TRUE

```
CREATE OR REPLACE PROCEDURE get_emps IS
  CURSOR get_emps_curs IS
  SELECT * FROM employees
  WHERE
  $IF tax_code_pack.new_tax_code $THEN
   salary > 20000;
  $ELSE
   salary > 50000;
  $END
BEGIN
  FOR v_emps IN get_emps_curs LOOP
  ... /* real code here */
  END LOOP;
END get_emps;
```

Using Selection Directives Example

- What's in the Data Dictionary?

```
SELECT text FROM USER_SOURCE
  WHERE type =
  'PROCEDURE' and name =
  'GET_EMPS'
  ORDER BY line;
```

TEXT
PROCEDURE get_emps IS
CURSOR get_emps_curs IS
SELECT * FROM employees
WHERE
$IF tax_code_pack.new_tax_code $THEN
salary > 2000
$END
BEGIN
FOR v_emps IN get_emps_curs LOOP
NULL; /* real code */
END LOOP;
END get_emps;

Using Selection Directives Example

- And what code was compiled?

```
BEGIN
  DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE
  ('PROCEDURE','<YOUR_USERNAME>','GET_EMPS');
END;
```

```
PROCEDURE get_emps IS
  CURSOR get_emps_curs IS
    SELECT * FROM employees
      WHERE

          salary > 20000;



BEGIN
  FOR v_emps IN get_emps_curs LOOP
    NULL; /* real code here */
  END LOOP;
END get_emps;
```

The PLSQL_CCFLAGS Initialization Parameter

- You may want to use the selection directives, such as $IF, to test for a condition that has nothing to do with global package constants or Oracle software versions
- For example, you may want to include extra code to help you debug a PL/SQL program, but once the errors have been corrected, you do not want to include this code in the final version because it will slow down the performance
- You can control this using the PLSQL_CCFLAGS initialization parameter

Oracle recommends that this parameter be used for controlling the conditional compilation of debugging or tracing code. This initialization parameter provides a mechanism that allows PL/SQL programmers to control conditional compilation of each PL/SQL library unit independently.

The PLSQL_CCFLAGS Parameter

- PLSQL_CCFLAGS allows you to set values for variables, and then test those variables in your PL/SQL program
- You define the variables and assign values to them using PLSQL_CCFLAGS
- Each PLSQL_CCFLAGS variable must be either a BOOLEAN literal (TRUE, FALSE, or NULL) or a PLS_INTEGER literal
- Then, you test the values of the variables in your PL/SQL program using inquiry directives
- An inquiry directive is the name of the variable prefixed by a double dollar sign ($$)

Using PLSQL_CCFLAGS and Inquiry Directives

- First, set the value of the parameter:

```
ALTER SESSION SET PLSQL_CCFLAGS = 'debug:true';
```

- Then compile your PL/SQL program:

```
CREATE OR REPLACE PROCEDURE testproc IS BEGIN
...
  $IF $$debug $THEN
  DBMS_OUTPUT.PUT_LINE('This code was executed');
  $END
...
END testproc;
```

Using PLSQL_CCFLAGS and Inquiry Directives

- After you have debugged the program, remove the debugging code by:

```
ALTER SESSION SET PLSQL_CCFLAGS = 'debug:false';
```

- Compile your procedure one more time to remove the debugging code

ORACLE Academy

Example:

ALTER SESSION SET PLSQL_CCFLAGS = 'DeBug:TruE';

ALTER SESSION SET PLSQL_CCFLAGS = 'debug:TRUE';

ALTER SESSION SET PLSQL_CCFLAGS = ‘testflag:1';

ALTER SESSION SET PLSQL_CCFLAGS = ‘TeStFlAg:1’;

Using PLSQL_CCFLAGS and Inquiry Directives

- DEBUG is not a keyword: you can use any name you like, and it can be a number or a character string, not just a Boolean

```
ALTER SESSION SET PLSQL_CCFLAGS = 'testflag:1';
```

```
CREATE OR REPLACE PROCEDURE testproc IS BEGIN
...
  $IF $$testflag > 0 $THEN
  DBMS_OUTPUT.PUT_LINE('This code was executed');
  $END
...
END testproc;
```

Using PLSQL_CCFLAGS and Inquiry Directives

- You can set more than one variable, and then test them either together or separately:

```
ALTER SESSION SET PLSQL_CCFLAGS =
  'firstflag:1, secondflag:false';
```

```
CREATE OR REPLACE PROCEDURE testproc IS BEGIN
...
  $IF $$firstflag > 0 AND NOT $$secondflag $THEN
  DBMS_OUTPUT.PUT_LINE('Testing both variables');
  $ELSIF $$secondflag $THEN
  DBMS_OUTPUT.PUT_LINE('Testing one variable');
  $END
...
END testproc;
```

Using PLSQL_CCFLAGS and Inquiry Directives

- You can see which settings of PLSQL_CCFLAGS were used to compile a program by querying USER_PLSQL_OBJECT_SETTINGS

```
SELECT name, plsql_ccflags
  FROM USER_PLSQL_OBJECT_SETTINGS
  WHERE name = 'TESTPROC';
```

NAME	PLSQL_CCFLAGS
TESTPROC	firstflag: 1, secondflag: false

This table displays information about the compiler settings for the stored objects owned by the current user.

NAME - VARCHAR2(30) NOT NULL Name of the object

TYPE - VARCHAR2(12) Type of the object: PROCEDURE, FUNCTION, PACKAGE, PACKAGE BODY, TRIGGER, TYPE, TYPE BODY

PLSQL_OPTIMIZE_LEVEL - NUMBER Optimization level that was used to compile the object

PLSQL_CODE_TYPE - VARCHAR2(4000) Compilation mode for the object

PLSQL_DEBUG - VARCHAR2(4000) Indicates whether the object was compiled with debug information or not

PLSQL_WARNINGS - VARCHAR2(4000) Compiler warning settings that were used to compile the object

NLS_LENGTH_SEMANTICS - VARCHAR2(4000) NLS length semantics that were used to compile the object

PLSQL_CCFLAGS - VARCHAR2(4000) Conditional compilation flag settings that were used to compile the object

PLSCOPE_SETTINGS - VARCHAR2(4000) Settings for using PL/Scope

Using PLSQL_CCFLAGS and Inquiry Directives

- And, as always, you can see what was included in the compiled program using DBMS_PREPROCESSOR

```
BEGIN
DBMS_PREPROCESSOR.PRINT_POST_PROCESSED_SOURCE
  ('PROCEDURE','<YOUR_USERNAME>','TESTPROC');
END;
```

```
PROCEDURE testproc IS BEGIN
--

    DBMS_OUTPUT.PUT_LINE('Testing both variables');



--
END testproc;
```

Using DBMS_DB_VERSION

- DBMS_DB_VERSION is a bodiless package that defines a number of constants, including:
  - VERSION (the current Oracle software version)
  - VER_LE_11 (= TRUE if the current Oracle software is version 11 or earlier)
  - VER_LE_10 (= TRUE if the current Oracle software is version 10 or earlier)
- So:

```
$IF DBMS_DB_VERSION.VER_LE_11 $THEN ...
```

- Is exactly the same as:

```
$IF DBMS_DB_VERSION.VERSION <= 11 $THEN ...
```


The DBMS_DB_VERSION package specifies the Oracle version numbers and other information useful for simple conditional compilation selections based on Oracle versions. The DBMS_DB_VERSION package contains different constants for different Oracle Database versions and releases.

VERSION - PLS_INTEGER with a value of 11: Current version

RELEASE - PLS_INTEGER with a value of 1: Current release

VER_LE_9 – BOOLEAN with a value of FALSE: Version <= 9

VER_LE_9_1 – BOOLEAN with a value of FALSE: Version <= 9 and release <= 1

VER_LE_9_2 – BOOLEAN with a value of FALSE: Version <= 9 and release <= 2 …

Terminology

- Key terms used in this lesson included:
  - Conditional compilation
  - DBMS_DB_VERSION
  - DBMS_PREPROCESSOR
  - Inquiry and selection directives
    - PLSQL_CCFLAGS
    - USER_SOURCE

ORACLE Academy

- Conditional Compilation - allows you to include some source code in your PL/SQL program that may be compiled or ignored by the compiler depending on a value checked at compilation.

- DBMS_DB_VERSION - package specifies the Oracle version numbers and other information useful for simple conditional compilation selections based on Oracle versions.

- DBMS_PREPROCESSOR - an Oracle-supplied package that can return or display the post-processed source text of a stored PL/SQL unit

- Inquiry and Selection Directives - used in conditional compilation

- PLSQL_CCFLAGS - mechanism that allows PL/SQL programmers to control conditional compilation of each PL/SQL library unit independently.

- USER_SOURCE - data dictionary view to see the complete source code, including the compiler directives,

for a stored PL/SQL unit

Summary

- In this lesson, you should have learned how to:
  - Describe the benefits of conditional compilation
  - Create and conditionally compile a PL/SQL program containing selection, inquiry, and error directives
  - Create and conditionally compile a PL/SQL program which calls the DBMS_DB_VERSION server-supplied package


ORACLE
Academy